

OSS Lab Exam

Complete Study Guide

Operating Systems & Security

19 Experiments | Viva Questions | Memory Tricks | Revision Sheet

Beginner-Friendly | Lab-Oriented | Exam-Ready

Exp	Topic	Key Concept
1	UNIX Commands	ls, cd, chmod, grep, ps
2	Fork()	Child/Parent process creation
3	Wait()	Zombie prevention, synchronization
4	Exec()	Process replacement
5	Stat()	File metadata retrieval
6	Readdir()	Directory listing
7	Round Robin	Time quantum, equal sharing
8	SJF	Sort by burst time, min WT
9	FCFS	Arrival order, convoy effect
10	Priority	Sort by priority, starvation
11	Producer-Consumer	Semaphores: empty, full, mutex
12	Shared Memory	shmget, shmat, shmdt
13	Banker's Algorithm	Need matrix, safe sequence
14	Deadlock Detection	Request matrix, finish array
15	FIFO Page Replace	Circular pointer, Belady's anomaly
16	LRU Page Replace	Last used time, no Belady's
17	LFU Page Replace	Access frequency count
18	Bell-LaPadula	Confidentiality: no read up, no write down
19	Biba Model	Integrity: no write up, no read down

Basics of UNIX Commands

■ 1. Aim

To learn and practice basic UNIX/Linux commands for file management, process control, and system navigation.

■ 2. Important Concept

UNIX is an operating system. Commands are typed in a terminal (shell). The shell reads your command, runs it, and shows output.

Think of it like talking to the computer in its language.

■ 3. Program Explanation (Line by Line)

```
ls          → List files and folders in current directory
ls -l       → Long listing: shows permissions, size, date
pwd         → Print Working Directory (where am I right now?)
cd /home    → Change Directory to /home
mkdir test  → Make a new folder named 'test'
rm file.txt → Remove/delete a file
cp a.txt b.txt → Copy a.txt to b.txt
mv a.txt /tmp → Move a.txt to /tmp folder
cat file.txt → Show contents of a file
chmod 777 file → Give full permissions to all users
ps aux      → Show all running processes
kill 1234    → Stop process with ID 1234
grep 'word' file → Search for 'word' in file
man ls       → Show manual/help for 'ls' command
who          → Show who is logged in
date         → Show current date and time
df -h        → Show disk space usage
wc -l file   → Count lines in a file
```

■ 4. How to Read in Viva

Section 1 - Navigation: pwd, ls, cd

Section 2 - File Operations: cp, mv, rm, mkdir, cat

Section 3 - Permissions: chmod (read=4, write=2, execute=1)

Section 4 - Process: ps, kill

Section 5 - Search: grep, wc, find

■ 5. Expected Output

```
$ pwd
/home/student
$ ls -l
```

```
total 8
-rw-r--r-- 1 student student 45 Jun 1 10:00 hello.txt
drwxr-xr-x 2 student student 4096 Jun 1 09:55 mydir
$ whoami
student
```

■ 6. Frequently Asked Viva Questions

Q1: What does chmod 755 mean?

A: Owner gets read+write+execute(7), Group gets read+execute(5), Others get read+execute(5).

Q2: Difference between cp and mv?

A: cp copies the file (original stays). mv moves/renames (original is gone).

Q3: What is a shell?

A: A shell is a program that reads your commands and passes them to the OS kernel to execute.

Q4: What does grep do?

A: grep searches for a pattern (text) inside a file and prints matching lines.

Q5: How to see all processes?

A: Use 'ps aux' or 'top' command.

■ 7. Memory Trick

Remember file permissions as 'rwx' = Read(4) Write(2) eXecute(1). 777=all allowed, 644=normal file, 755=normal program.

■ 8. Common Errors and Fixes

Error: Permission denied

Fix: Use sudo before command, or chmod to set correct permissions.

Error: No such file or directory

Fix: Check spelling. Use ls to confirm file exists. Use pwd to check current directory.

Error: Command not found

Fix: Install the package, or check if you typed the command correctly.

■ 9. Exam Writing Steps (Algorithm)

1. Start with pwd to show current location
2. Use ls to list files
3. mkdir to create directory
4. cd to enter directory
5. touch/cat to create and view files
6. chmod to set permissions

■ 10. 3 Marks Summary

UNIX commands control files, processes, and the system. Key commands: ls (list), cd (change dir), cp (copy), mv (move), rm (remove), chmod (permissions), grep (search), ps (processes), kill (stop process). Permissions use numbers: r=4, w=2, x=1.

Fork System Call

■ 1. Aim

To demonstrate process creation using the fork() system call in C.

■ 2. Important Concept

fork() creates a new process (child) that is an exact copy of the current process (parent).

After fork():

- In PARENT: fork() returns the child's PID (a positive number)
- In CHILD: fork() returns 0
- On ERROR: fork() returns -1

It's like photocopying yourself — now two of you exist and do different things.

■ 3. Program Explanation (Line by Line)

```
#include <stdio.h>      → include input/output library
#include <unistd.h>     → include Unix system call library (for fork)
#include <sys/types.h> → include types like pid_t

int main() {
    pid_t pid = fork(); // CREATE A NEW PROCESS HERE ← VIVA LINE

    if (pid < 0) {      // fork failed
        printf("Fork failed!\n");
        return 1;
    }

    else if (pid == 0) { // We are in CHILD process
        printf("CHILD: My PID = %d\n", getpid());
        printf("CHILD: My Parent PID = %d\n", getppid());
    }

    else {              // We are in PARENT process
        printf("PARENT: My PID = %d\n", getpid());
        printf("PARENT: Child PID = %d\n", pid);
    }

    return 0;
}
```

■ 4. How to Read in Viva

Line 1-3: Include necessary headers

Line 5: pid_t pid = fork() — this is the KEY line; creates child process

Lines 7-11: Error check (pid < 0)

Lines 12-15: Child process code (pid == 0)

Lines 16-20: Parent process code (pid > 0)

■ 5. Expected Output

```
PARENT: My PID = 1234
PARENT: Child PID = 1235
CHILD: My PID = 1235
CHILD: My Parent PID = 1234
(Note: Order may vary – OS decides who runs first)
```

■ 6. Frequently Asked Viva Questions

Q1: What does fork() return?

A: Returns child PID to parent, 0 to child, -1 on error.

Q2: What is PID?

A: Process ID — a unique number assigned to every running process.

Q3: What is getpid() and getppid()?

A: getpid() returns current process's PID. getppid() returns parent's PID.

Q4: Is the child process an exact copy?

A: Yes, child gets a copy of parent's code, data, and stack. But they have different PIDs.

Q5: What happens if fork() fails?

A: It returns -1. Usually happens when system has too many processes (resource limit hit).

■ 7. Memory Trick

Fork returns ZERO to child, POSITIVE to parent. Remember: Child=0 (child is junior/zero), Parent=positive (parent is boss with positive number).

■ 8. Common Errors and Fixes

Error: Forgetting to include unistd.h

Fix: Add #include at the top.

Error: Not checking return value

Fix: Always check if pid < 0 (error), == 0 (child), > 0 (parent).

Error: Zombie process

Fix: Parent should call wait() so child's exit status is collected.

■ 9. Exam Writing Steps (Algorithm)

1. Include headers: stdio.h, unistd.h, sys/types.h
2. Declare pid_t pid
3. Call pid = fork()
4. Check if pid < 0 (error)
5. Check if pid == 0 (child code)
6. Else: parent code

■ 10. 3 Marks Summary

fork() creates a child process that is a copy of the parent. Returns 0 to child, positive PID to parent, -1 on error. Both processes run concurrently after fork(). Use getpid() for own PID and getppid() for parent PID.

Wait System Call

■ 1. Aim

To demonstrate process synchronization using the wait() system call.

■ 2. Important Concept

wait() makes the PARENT wait/pause until the CHILD process finishes.

Without wait(): parent may finish first, leaving child as a zombie.

With wait(): parent waits for child → child finishes → parent continues.

Think of it like a teacher waiting for a student to finish their test.

■ 3. Program Explanation (Line by Line)

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h> // needed for wait() ← IMPORTANT
#include <sys/types.h>

int main() {
    pid_t pid = fork(); // create child process

    if (pid == 0) {      // CHILD process
        printf("Child is running...\n");
        sleep(2);       // child sleeps for 2 seconds
        printf("Child finished.\n");
        return 0;
    }
    else {               // PARENT process
        printf("Parent waiting for child...\n");
        wait(NULL);      // PARENT WAITS HERE ← KEY LINE
        printf("Parent: child has finished!\n");
    }
    return 0;
}
```

■ 4. How to Read in Viva

Line 3: sys/wait.h is the header for wait()

Line 7: fork() creates the child

Lines 9-13: Child process — runs its task, then exits

Lines 14-18: Parent calls wait(NULL) — pauses until child finishes

After wait() returns: parent knows child is done

■ 5. Expected Output

```
Parent waiting for child...
```

```
Child is running...
Child finished.
Parent: child has finished!
```

■ 6. Frequently Asked Viva Questions

Q1: What is a zombie process?

A: A process that has finished but its entry still exists in the process table because parent hasn't called wait().

Q2: What does wait(NULL) mean?

A: Parent waits for any child to finish. NULL means we don't need the exit status.

Q3: What is sys/wait.h?

A: A header file that contains the wait() function declaration.

Q4: What is an orphan process?

A: When the parent finishes before the child. The child is adopted by init (PID 1).

Q5: How does wait() help?

A: It prevents zombie processes and ensures parent knows when child has finished.

■ 7. Memory Trick

WAIT = 'W'atch 'A'nd 'I'dle 'T'ill child ends. Parent just waits and does nothing until child says 'I'm done!'

■ 8. Common Errors and Fixes

Error: Forgetting sys/wait.h

Fix: Add #include otherwise wait() won't compile.

Error: Calling wait() without fork()

Fix: wait() only makes sense in a process that has children.

Error: Zombie processes

Fix: Always call wait() in parent to clean up child processes.

■ 9. Exam Writing Steps (Algorithm)

1. Include stdio.h, unistd.h, sys/wait.h, sys/types.h
2. Call fork() to create child
3. In child section: do child's work, then return 0
4. In parent section: call wait(NULL)
5. Print message after wait() to confirm child is done

■ 10. 3 Marks Summary

wait() suspends the parent process until one of its children terminates. Prevents zombie processes. Include sys/wait.h. wait(NULL) ignores exit status. waitpid() waits for specific child. Without wait(), child may become zombie.

Exec System Call

1. Aim

To demonstrate how `exec()` replaces the current process image with a new program.

2. Important Concept

`exec()` completely REPLACES the current process with a new program.

Unlike `fork()` which creates a copy, `exec()` overwrites the current process.

After `exec()` succeeds, the code after it NEVER runs because the old program is gone.

Think of it like: you enter a room and transform completely into someone else.

3. Program Explanation (Line by Line)

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Before exec\n");    // this prints

    // Replace current process with 'ls' command
    execl("/bin/ls", "ls", "-l", NULL); // ← KEY LINE

    // If exec succeeds, this line NEVER executes
    printf("After exec\n");    // this will NOT print if exec works

    return 0;
}

// Fork + Exec Example (better practice):
// pid_t pid = fork();
// if (pid == 0) {
//     execl('/bin/ls', 'ls', NULL);
// }
// wait(NULL);
```

4. How to Read in Viva

Line 5: Prints before `exec` — this runs normally

Line 8: `execl()` — replaces the process with `/bin/ls`

- First arg: full path of program (`/bin/ls`)
- Second arg: name of program (`ls`)
- Further args: command arguments (`-l`)
- Last arg: `NULL` (always required to end the list)

Line 10-11: These lines NEVER run if `exec` succeeds

5. Expected Output

Before exec


```
total 24
-rw-r--r-- 1 student student 234 Jun 1 myfile.c
-rwxr-xr-x 1 student student 8192 Jun 1 a.out
(Note: 'After exec' is NEVER printed)
```

■ 6. Frequently Asked Viva Questions

Q1: What does `exec()` do?

A: It replaces the current process's code with a new program. The PID remains the same.

Q2: Does `exec()` create a new process?

A: No! It replaces the existing one. Use `fork()` first if you want to keep the original.

Q3: What happens if `exec()` fails?

A: Execution continues to the next line after `exec()`. Returns -1 on failure.

Q4: Difference between `execl()` and `execv()`?

A: `execl()` takes arguments as a list. `execv()` takes arguments as an array (vector).

Q5: Why use `fork()` before `exec()`?

A: So the original process (parent) continues after `exec` replaces the child process.

■ 7. Memory Trick

exec = EXECute and replace. Remember: exec 'eats' the process. The old program is gone! Always use fork() first if you want to survive after exec().

■ 8. Common Errors and Fixes

Error: `exec()` not replacing process

Fix: Check the path. `/bin/ls` must exist. Use `which ls` to find path.

Error: NULL not at end of `execl` args

Fix: Always end the argument list with NULL.

Error: Code after `exec()` runs

Fix: This means `exec()` failed. Print `perror()` to see why.

■ 9. Exam Writing Steps (Algorithm)

1. Include `stdio.h` and `unistd.h`
2. Print message before `exec`
3. Call `execl()` with full path, program name, args, NULL
4. Show that code after `exec` doesn't run (to prove `exec` worked)
5. For safe usage: `fork()` first, `exec()` in child

■ 10. 3 Marks Summary

`exec()` replaces the current process image with a new program. Returns only on failure. The PID stays the same but code changes. `execl()` uses list args; `execv()` uses array args. Always end argument list with NULL. Usually combined with `fork()` in practice.

Stat System Call

■ 1. Aim

To retrieve and display file metadata (size, permissions, timestamps) using the stat() system call.

■ 2. Important Concept

stat() reads a file's metadata — information ABOUT the file, not the file's content.

Metadata = file size, owner, permissions, last modified time, inode number, etc.

Every file in UNIX has a structure called 'struct stat' that holds all this info.

■ 3. Program Explanation (Line by Line)

```
#include <stdio.h>
#include <sys/stat.h> // for stat() and struct stat
#include <sys/types.h>
#include <time.h>      // for ctime() to print time

int main() {
    struct stat info;    // structure to store file info
    char *filename = "test.txt";

    if (stat(filename, &info) != 0) { // call stat() ← KEY LINE
        printf("Error reading file info\n");
        return 1;
    }

    printf("File: %s\n", filename);
    printf("Size: %ld bytes\n", info.st_size);    // file size
    printf("Inode: %ld\n", info.st_ino);          // inode number
    printf("Hard links: %ld\n", info.st_nlink);   // link count
    printf("Permissions: %o\n", info.st_mode);   // octal permissions
    printf("Last modified: %s", ctime(&info.st_mtime)); // time
    return 0;
}
```

■ 4. How to Read in Viva

Line 2: sys/stat.h — required header for stat()

Line 6: struct stat info — the container for all file metadata

Line 10: stat(filename, &info;) — fills the 'info' structure

Lines 15-19: Access individual fields of the struct:

st_size = size in bytes

st_ino = inode number

st_mode = file type and permissions

st_mtime= last modification time

■ 5. Expected Output

```
File: test.txt
Size: 1024 bytes
Inode: 123456
Hard links: 1
Permissions: 100644
Last modified: Mon Jun  1 10:30:00 2025
```

■ 6. Frequently Asked Viva Questions

Q1: What is an inode?

A: An inode is a data structure that stores metadata about a file (not its name or content).

Q2: What is st_mode?

A: st_mode stores file type and permission bits in octal format.

Q3: What header is needed for stat()?

A: sys/stat.h is the required header.

Q4: Difference between stat() and lstat()?

A: stat() follows symbolic links. lstat() returns info about the link itself, not the target.

Q5: What is st_mtime?

A: st_mtime is the time when the file content was last modified.

■ 7. Memory Trick

STAT = 'STATistics of a file'. Remember the key fields: st_SIZE, st_INO, st_MODE, st_MTIME. Think: 'SIMM' — Size, Inode, Mode, Mtime.

■ 8. Common Errors and Fixes

Error: stat() returns -1

Fix: File may not exist. Check filename spelling. Use perror() to see error.

Error: Wrong format specifier for st_size

Fix: Use %ld (long) not %d (int) for stat fields.

Error: Missing sys/stat.h

Fix: Always include sys/stat.h for stat() to work.

■ 9. Exam Writing Steps (Algorithm)

1. Include sys/stat.h, stdio.h, time.h
2. Declare struct stat variable
3. Call stat(filename, &struct;_var)
4. Check return value (0=success, -1=error)
5. Print required fields: st_size, st_ino, st_mode, st_mtime

■ 10. 3 Marks Summary

stat() retrieves file metadata and stores it in struct stat. Key fields: st_size (bytes), st_ino (inode), st_mode (permissions/type), st_mtime (mod time), st_nlink (hard links). Returns 0 on success, -1 on failure. Header: sys/stat.h.

Readdir System Call

■ 1. Aim

To list the contents of a directory using opendir() and readdir() system calls.

■ 2. Important Concept

readdir() reads one entry (file/subdirectory) at a time from an open directory.

Steps: opendir() → readdir() in loop → closedir()

Each call to readdir() returns a pointer to a struct dirent with the file name.

■ 3. Program Explanation (Line by Line)

```
#include <stdio.h>
#include <dirent.h>    // for DIR, dirent, opendir, readdir

int main() {
    DIR *dir;           // directory pointer
    struct dirent *entry; // one directory entry
    char *path = ".";    // current directory

    dir = opendir(path); // open directory ← KEY
    if (dir == NULL) {
        printf("Cannot open directory\n");
        return 1;
    }

    printf("Files in directory:\n");
    while ((entry = readdir(dir)) != NULL) { // read each entry
        printf(" %s\n", entry->d_name); // print file name
    }

    closedir(dir); // close the directory ← always close!
    return 0;
}
```

■ 4. How to Read in Viva

Line 2: dirent.h — header for directory operations

Line 5: DIR *dir — directory stream pointer

Line 6: struct dirent *entry — holds one file entry at a time

Line 9: opendir(path) — opens the directory, like fopen for files

Line 15-17: Loop reads one entry at a time until NULL (no more entries)

Line 17: entry->d_name — the name of the file/folder

Line 19: closedir(dir) — always close, like fclose

■ 5. Expected Output

Files in directory:

```
.  
..  
main.c  
a.out  
test.txt  
mydir
```

■ 6. Frequently Asked Viva Questions

Q1: What does opendir() return?

A: Returns a DIR pointer (directory stream) on success, NULL on failure.

Q2: What is struct dirent?

A: A structure that holds info about one directory entry. d_name holds the filename.

Q3: Why do we see '.' and '..' in output?

A: '.' means current directory, '..' means parent directory. They are always present.

Q4: How do we stop the readdir() loop?

A: readdir() returns NULL when there are no more entries. The while loop exits.

Q5: What header is required for readdir()?

A: dirent.h is required for DIR, struct dirent, opendir, readdir, closedir.

■ 7. Memory Trick

DIR operations follow the same pattern as file operations: OPEN → READ in loop → CLOSE. Replace fopen with opendir, fread with readdir, fclose with closedir.

■ 8. Common Errors and Fixes

Error: opendir() returns NULL

Fix: Directory path doesn't exist. Check path. Use '.' for current dir.

Error: Forgetting closedir()

Fix: Always call closedir() to free resources.

Error: Infinite loop

Fix: Ensure the while condition uses readdir() correctly.

■ 9. Exam Writing Steps (Algorithm)

1. Include dirent.h and stdio.h
2. Declare DIR *dir and struct dirent *entry
3. Call opendir() and check for NULL
4. Loop: while(readdir()) != NULL) print entry->d_name
5. Call closedir()

■ 10. 3 Marks Summary

readdir() reads directory entries one by one. Flow: opendir() → while(readdir()) → closedir(). struct dirent holds d_name (filename) and d_type (type). Header: dirent.h. Always close directory with closedir(). '.' and '..' are always listed.

Round Robin Scheduling Algorithm

■ 1. Aim

To implement Round Robin CPU scheduling algorithm with a given time quantum.

■ 2. Important Concept

Round Robin = Fair sharing. Each process gets equal CPU time (time quantum/slice).

If a process doesn't finish in its quantum, it goes back to the END of the queue.

Used in time-sharing systems. No starvation. Fair to all processes.

Key formula: If burst time > quantum → remaining = burst - quantum

■ 3. Program Explanation (Line by Line)

```
#include <stdio.h>

int main() {
    int n, quantum;
    printf("Enter number of processes: "); scanf("%d", &n);
    printf("Enter time quantum: "); scanf("%d", &quantum);

    int bt[n], remaining[n], wt[n], tat[n];
    for(int i=0; i<n; i++) {
        printf("Burst time P%d: ", i+1); scanf("%d", &bt[i]);
        remaining[i] = bt[i]; // copy burst time to remaining
    }

    int time=0, done=0;
    while(done < n) {          // keep going until all done
        done = 0;
        for(int i=0; i<n; i++) {
            if(remaining[i] > 0) { // process not yet finished
                if(remaining[i] > quantum) {
                    time += quantum;
                    remaining[i] -= quantum; // reduce by quantum
                } else {
                    time += remaining[i];
                    tat[i] = time;           // TAT = finish time
                    wt[i] = tat[i] - bt[i]; // WT = TAT - BT
                    remaining[i] = 0;
                }
            } else done++;
        }
    }

    float avgWT=0, avgTAT=0;
    for(int i=0; i<n; i++) { avgWT+=wt[i]; avgTAT+=tat[i]; }
```

```
printf("Avg WT: %.2f, Avg TAT: %.2f\n", avgWT/n, avgTAT/n);
return 0;
}
```

■ 4. How to Read in Viva

Section 1 (lines 4-11): Input — get burst times, copy to remaining[]

Section 2 (lines 13-29): Round Robin logic

- Outer while: runs until all processes done
- Inner for: gives each process one quantum at a time
- If remaining > quantum: subtract quantum, continue loop
- If remaining <= quantum: process finishes, record TAT and WT

Section 3 (lines 30-32): Calculate and print averages

■ 5. Expected Output

```
Enter number of processes: 3
Enter time quantum: 2
Burst time P1: 5
Burst time P2: 3
Burst time P3: 1
P  BT  WT  TAT
P1  5   6  11
P2  3   5   8
P3  1   4   5
Avg WT: 5.00, Avg TAT: 8.00
```

■ 6. Frequently Asked Viva Questions

Q1: What is time quantum?

A: A fixed time slice given to each process. If process doesn't finish, it goes back to queue.

Q2: What is the formula for Waiting Time?

A: $WT = \text{Turnaround Time} - \text{Burst Time}$

Q3: What is Turnaround Time?

A: $TAT = \text{Completion Time} - \text{Arrival Time}$ (if arrival=0, TAT = completion time)

Q4: Does Round Robin cause starvation?

A: No! Every process gets equal CPU time, so no process waits forever.

Q5: What happens if quantum is very large?

A: It becomes like FCFS (each process runs to completion before switching).

■ 7. Memory Trick

*RR = Round like a circle. Think of a carousel — every child gets one turn, then back to the end of line.
Quantum = how long each ride lasts.*

■ 8. Common Errors and Fixes

Error: remaining[] not initialized

Fix: Always copy `bt[i]` to `remaining[i]` before the scheduling loop.

Error: Infinite loop

Fix: Check that done counter resets to 0 at start of each outer loop iteration.

Error: Wrong TAT formula

Fix: $TAT = \text{completion time}$. $WT = TAT - \text{burst time}$.

■ 9. Exam Writing Steps (Algorithm)

1. Get n processes and time quantum
2. Get burst times, initialize `remaining[] = bt[]`
3. Loop: for each process with `remaining > 0`
4. If `remaining > quantum`: `time += quantum`, `remaining -= quantum`
5. Else: `time += remaining`, compute TAT and WT, set `remaining = 0`
6. Print table and averages

■ 10. 3 Marks Summary

Round Robin gives each process equal CPU time (quantum). No starvation. $WT = TAT - BT$. $TAT = \text{finish time} - \text{arrival time}$. Used in time-sharing OS. Key: `remaining[]` array tracks unfinished work. If quantum too small = too many context switches; too large = like FCFS.

SJF Scheduling Algorithm (Non-Preemptive)

1. Aim

To implement Shortest Job First (SJF) scheduling to minimize average waiting time.

2. Important Concept

SJF = Shortest Job First. The process with the SMALLEST burst time runs first.

Like a doctor treating patients: someone with a 2-minute checkup goes before a 1-hour surgery.

NON-PREEMPTIVE: once started, a process runs to completion.

SJF gives the MINIMUM average waiting time (optimal for non-preemptive).

3. Program Explanation (Line by Line)

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter number of processes: "); scanf("%d", &n);
    int bt[n], wt[n], tat[n], idx[n];
    for(int i=0; i<n; i++) {
        printf("Burst time P%d: ", i+1); scanf("%d", &bt[i]);
        idx[i] = i; // track original process number
    }

    // Sort by burst time (Selection Sort) ← KEY SECTION
    for(int i=0; i<n-1; i++) {
        int min=i;
        for(int j=i+1; j<n; j++)
            if(bt[idx[j]] < bt[idx[min]]) min=j;
        int temp=idx[i]; idx[i]=idx[min]; idx[min]=temp;
    }

    // Calculate WT and TAT
    wt[0] = 0; // first process waits 0
    for(int i=1; i<n; i++)
        wt[i] = wt[i-1] + bt[idx[i-1]]; // wait = sum of all previous BTs
    for(int i=0; i<n; i++)
        tat[i] = wt[i] + bt[idx[i]]; // TAT = WT + BT

    float avgWT=0, avgTAT=0;
    for(int i=0; i<n; i++) { avgWT+=wt[i]; avgTAT+=tat[i]; }
    printf("Avg WT: %.2f, Avg TAT: %.2f\n", avgWT/n, avgTAT/n);
    return 0;
}
```

4. How to Read in Viva

Section 1: Input burst times, initialize idx[] for sorting

Section 2: Sort processes by burst time (shortest first)

- Uses selection sort on idx[] array

Section 3: WT calculation

- First process: $wt[0] = 0$
- Others: $wt[i] = wt[i-1] + bt[sorted[i-1]]$

Section 4: $TAT = WT + Burst\ Time$

■ 5. Expected Output

```
Enter number of processes: 3
Burst time P1: 6
Burst time P2: 2
Burst time P3: 4
Sorted Order: P2(2) → P3(4) → P1(6)
P  BT  WT  TAT
P2  2   0   2
P3  4   2   6
P1  6   6  12
Avg WT: 2.67, Avg TAT: 6.67
```

■ 6. Frequently Asked Viva Questions

Q1: Why is SJF optimal?

A: SJF gives minimum average waiting time among all non-preemptive algorithms.

Q2: What is the main disadvantage of SJF?

A: Starvation — long processes may wait forever if short processes keep arriving.

Q3: Preemptive SJF has another name?

A: SRTF — Shortest Remaining Time First.

Q4: When is arrival time 0 for all?

A: $wt[0]=0$ and each subsequent wt = sum of all previous burst times.

Q5: What is the key difference from Round Robin?

A: SJF runs the shortest job completely first. RR gives equal time slices to all.

■ 7. Memory Trick

SJF = 'Smallest job goes First'. Sort the jobs by size, smallest first, then run them in that order. Like sorting M&Ms; by size and eating smallest first!

■ 8. Common Errors and Fixes

Error: Not sorting before calculating WT

Fix: WT depends on execution order. Sort first, then compute WT.

Error: Not tracking original process index

Fix: Use idx[] array to remember which process was which after sorting.

Error: Wrong WT formula

Fix: $wt[i] = wt[i-1] + bt$ of SORTED process i-1, not original i-1.

■ 9. Exam Writing Steps (Algorithm)

1. Get burst times for all processes

2. Sort processes by burst time (ascending order)
3. $wt[0] = 0$
4. $wt[i] = wt[i-1] + bt[i-1]$ (after sorting)
5. $tat[i] = wt[i] + bt[i]$
6. Average WT = $\text{sum}(wt)/n$, Average TAT = $\text{sum}(tat)/n$

■ 10. 3 Marks Summary

SJF selects process with smallest burst time first. Optimal average WT. Problem: starvation of long processes. Sort by BT, then $WT[0]=0$, $WT[i]=WT[i-1]+BT[i-1]$. $TAT=WT+BT$. Preemptive version is SRTF.

FCFS Scheduling Algorithm

■ 1. Aim

To implement First Come First Serve (FCFS) CPU scheduling algorithm.

■ 2. Important Concept

FCFS = First Come First Serve. Simple queue — whoever arrives first, gets CPU first.

Like a bus stop: first person in line boards first.

Non-preemptive. Easy to implement. But can cause 'Convoy Effect' — short jobs wait for long ones.

■ 3. Program Explanation (Line by Line)

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter number of processes: "); scanf("%d", &n);
    int bt[n], at[n], wt[n], tat[n];
    for(int i=0; i<n; i++) {
        printf("Arrival time P%d: ", i+1); scanf("%d", &at[i]);
        printf("Burst time P%d: ", i+1); scanf("%d", &bt[i]);
    }

    // FCFS: processes run in order of arrival
    int time = 0;
    for(int i=0; i<n; i++) {
        if(time < at[i]) time = at[i]; // CPU might be idle
        wt[i] = time - at[i];          // WT = start - arrival
        time += bt[i];                 // advance time by burst
        tat[i] = time - at[i];         // TAT = finish - arrival
    }

    float avgWT=0, avgTAT=0;
    for(int i=0; i<n; i++) { avgWT+=wt[i]; avgTAT+=tat[i]; }
    printf("Avg WT: %.2f\n", avgWT/n);
    return 0;
}
```

■ 4. How to Read in Viva

The key calculation lines:

Line 14: `if(time < at[i]) time=at[i]` → handles CPU idle time between processes

Line 15: `wt[i] = time - at[i]` → waiting time = when CPU available minus when arrived

Line 16: `time += bt[i]` → move clock forward by process's burst time

Line 17: `tat[i] = time - at[i]` → TAT = finish time minus arrival time

■ 5. Expected Output

```
Enter number of processes: 3
P1: AT=0, BT=5 | P2: AT=1, BT=3 | P3: AT=2, BT=1
P  AT  BT  WT  TAT
P1  0   5   0   5
P2  1   3   4   7
P3  2   1   6   7
Avg WT: 3.33, Avg TAT: 6.33
```

■ 6. Frequently Asked Viva Questions

Q1: What is the Convoy Effect?

A: Short processes wait behind one long process, like small cars stuck behind a slow truck.

Q2: Is FCFS preemptive or non-preemptive?

A: Non-preemptive. Once a process starts, it runs until it finishes.

Q3: What is the formula for waiting time in FCFS?

A: $WT = \text{Start Time} - \text{Arrival Time}$

Q4: What happens if all processes arrive at the same time?

A: They are served in the order they are entered (no meaningful difference from order).

Q5: Advantage of FCFS?

A: Simple to implement and understand. Fair in the sense that whoever comes first is served first.

■ 7. Memory Trick

FCFS = just a queue (like WhatsApp message order). Calculate in ORDER: first process waits 0, each next waits for all previous to finish.

■ 8. Common Errors and Fixes

Error: Not handling CPU idle time

Fix: If $at[i] > \text{current time}$, set $\text{time} = at[i]$ before calculating WT.

Error: Mixing up WT and TAT formulas

Fix: $WT = \text{start} - \text{arrival}$. $TAT = \text{finish} - \text{arrival} = WT + BT$.

Error: Not sorting by arrival time

Fix: If processes arrive out of order, sort by arrival time first.

■ 9. Exam Writing Steps (Algorithm)

1. Get arrival time and burst time for each process
2. Sort by arrival time
3. $\text{time} = 0$ (system clock)
4. For each process: check CPU idle, compute $WT = \text{time} - AT$
5. Advance: $\text{time} += BT$
6. $TAT = \text{time} - AT = WT + BT$
7. Print averages

■ 10. 3 Marks Summary

FCFS serves processes in arrival order. Non-preemptive. Simple but suffers convoy effect. $WT = \text{start time} - \text{arrival time}$. $TAT = \text{completion} - \text{arrival} = WT + BT$. No starvation but poor average WT compared to SJF.

Priority Scheduling Algorithm

1. Aim

To implement Priority Scheduling where CPU is allocated to the highest priority process.

2. Important Concept

Priority Scheduling: each process has a PRIORITY NUMBER. Lowest number = Highest priority (usually).

Process with highest priority runs first.

Problem: STARVATION — low priority processes may never get CPU if high priority ones keep coming.

Solution: AGING — increase priority of waiting processes over time.

3. Program Explanation (Line by Line)

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter number of processes: "); scanf("%d", &n);
    int bt[n], pr[n], wt[n], tat[n], idx[n];
    for(int i=0; i<n; i++) {
        printf("Burst time P%d: ", i+1); scanf("%d", &bt[i]);
        printf("Priority P%d: ", i+1); scanf("%d", &pr[i]);
        idx[i] = i;
    }

    // Sort by priority (lower number = higher priority) ← KEY
    for(int i=0; i<n-1; i++) {
        int min=i;
        for(int j=i+1; j<n; j++)
            if(pr[idx[j]] < pr[idx[min]]) min=j; // find lowest priority number
        int t=idx[i]; idx[i]=idx[min]; idx[min]=t;
    }

    wt[0]=0;
    for(int i=1; i<n; i++)
        wt[i] = wt[i-1] + bt[idx[i-1]];
    for(int i=0; i<n; i++)
        tat[i] = wt[i] + bt[idx[i]];

    float avgWT=0;
    for(int i=0; i<n; i++) avgWT+=wt[i];
    printf("Avg WT: %.2f\n", avgWT/n);
    return 0;
}
```

4. How to Read in Viva

Same structure as SJF — the ONLY difference:

SJF sorts by: smallest burst time first

Priority sorts by: smallest priority NUMBER first (= highest priority)

The sorting, WT, TAT calculations are identical after sorting.

■ 5. Expected Output

Enter number of processes: 3

P1: BT=5, Priority=3 | P2: BT=3, Priority=1 | P3: BT=1, Priority=2

Execution Order: P2(pri=1) → P3(pri=2) → P1(pri=3)

P	BT	PR	WT	TAT
---	----	----	----	-----

P2	3	1	0	3
----	---	---	---	---

P3	1	2	3	4
----	---	---	---	---

P1	5	3	4	9
----	---	---	---	---

Avg WT: 2.33, Avg TAT: 5.33

■ 6. Frequently Asked Viva Questions

Q1: What causes starvation in Priority Scheduling?

A: High-priority processes keep arriving, so low-priority processes never get CPU.

Q2: What is Aging?

A: Gradually increasing the priority of processes that wait for a long time, to prevent starvation.

Q3: Is lower number = higher or lower priority?

A: Convention: lower number = higher priority (priority 1 > priority 5).

Q4: Difference between Priority and SJF?

A: SJF sorts by burst time. Priority sorts by priority number. Structure of code is the same.

Q5: What is preemptive priority scheduling?

A: If a new high-priority process arrives, it preempts (interrupts) the current process.

■ 7. Memory Trick

*Priority = VIP queue. Priority 1 is the President, Priority 5 is the intern. President always goes first!
Remember: LOW number = HIGH importance.*

■ 8. Common Errors and Fixes

Error: Sorting in wrong order

Fix: Sort ascending by priority number (smallest first = highest priority runs first).

Error: Confusion about priority convention

Fix: In most exam problems, smaller number = higher priority. Confirm in question.

Error: Starvation not explained

Fix: In viva, always mention starvation as a disadvantage and aging as solution.

■ 9. Exam Writing Steps (Algorithm)

1. Get burst time and priority for each process
2. Sort by priority number (ascending = highest priority first)
3. $wt[0] = 0$
4. $wt[i] = wt[i-1] + bt[sorted_i-1]$
5. $tat[i] = wt[i] + bt[i]$
6. Print and average

■ 10. 3 Marks Summary

Priority scheduling runs highest priority (lowest number) process first. Non-preemptive version sorts by priority then runs in order. Problem: starvation. Solution: aging. Structure is same as SJF — just sort by priority instead of burst time.

Producer Consumer Problem Using Semaphores

1. Aim

To solve the Producer-Consumer synchronization problem using POSIX semaphores.

2. Important Concept

Producer creates items and puts them in a SHARED BUFFER.

Consumer takes items from the buffer.

Problem: Producer shouldn't add when buffer is FULL. Consumer shouldn't take when buffer is EMPTY.

Solution: Use SEMAPHORES to synchronize.

- empty semaphore: counts empty slots (starts at BUFFER_SIZE)
- full semaphore: counts filled slots (starts at 0)
- mutex: ensures only one accesses buffer at a time

3. Program Explanation (Line by Line)

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE], in=0, out=0;
sem_t empty, full, mutex;    // THREE semaphores ← KEY

void* producer(void* arg) {
    for(int i=1; i<=5; i++) {
        sem_wait(&empty);    // wait if buffer is full
        sem_wait(&mutex);    // lock buffer
        buffer[in] = i;      // produce item
        in = (in+1) % BUFFER_SIZE;
        printf("Produced: %d\n", i);
        sem_post(&mutex);    // unlock buffer
        sem_post(&full);     // signal: one more item available
    }
}

void* consumer(void* arg) {
    for(int i=1; i<=5; i++) {
        sem_wait(&full);     // wait if buffer is empty
        sem_wait(&mutex);    // lock buffer
        int item = buffer[out];
        out = (out+1) % BUFFER_SIZE;
        printf("Consumed: %d\n", item);
        sem_post(&mutex);    // unlock buffer
        sem_post(&empty);    // signal: one more space available
    }
}
```

```

    }
}

int main() {
    sem_init(&empty, 0, BUFFER_SIZE); // empty starts full
    sem_init(&full, 0, 0);             // full starts empty
    sem_init(&mutex, 0, 1);            // mutex starts at 1 (unlocked)
    pthread_t pt, ct;
    pthread_create(&pt, NULL, producer, NULL);
    pthread_create(&ct, NULL, consumer, NULL);
    pthread_join(pt, NULL);
    pthread_join(ct, NULL);
    return 0;
}

```

■ 4. How to Read in Viva

Three semaphores are KEY:

empty: initialized to BUFFER_SIZE (buffer has 5 empty slots initially)

full: initialized to 0 (no items initially)

mutex: initialized to 1 (acts like a binary lock)

Producer sequence: wait(empty) → wait(mutex) → produce → post(mutex) → post(full)

Consumer sequence: wait(full) → wait(mutex) → consume → post(mutex) → post(empty)

Order matters: always lock mutex AFTER the capacity semaphore to avoid deadlock

■ 5. Expected Output

```

Produced: 1
Consumed: 1
Produced: 2
Produced: 3
Consumed: 2
Consumed: 3
(Order may vary due to thread scheduling)

```

■ 6. Frequently Asked Viva Questions

Q1: What is a semaphore?

A: A variable used for process synchronization. sem_wait() decrements it (blocks if 0). sem_post() increments it.

Q2: Why do we need three semaphores?

A: empty: tracks free slots, full: tracks filled slots, mutex: prevents simultaneous access.

Q3: What happens without mutex?

A: Both producer and consumer may access buffer at the same time, causing race condition.

Q4: What is a circular buffer?

A: Buffer where in/out pointers wrap around using modulo: $(in+1) \% BUFFER_SIZE$.

Q5: Compile command?

A: gcc program.c -lpthread -o output (need -lpthread flag for threads)

■ 7. Memory Trick

Remember PC semaphore order using: 'EFM' — Empty, Full, Mutex. Producer: wait Empty, wait Mutex, PUT, post Mutex, post Full. Consumer: wait Full, wait Mutex, GET, post Mutex, post Empty.

■ 8. Common Errors and Fixes

Error: Forgetting -lpthread in compile

Fix: Always compile with: `gcc file.c -lpthread`

Error: Wrong init values

Fix: `empty=BUFFER_SIZE, full=0, mutex=1`

Error: Deadlock: locking mutex before capacity semaphore

Fix: Always wait on empty/full BEFORE mutex.

■ 9. Exam Writing Steps (Algorithm)

1. Define `BUFFER_SIZE`, `buffer[]`, in, out
2. Declare `sem_t empty, full, mutex`
3. Initialize: `empty=SIZE, full=0, mutex=1`
4. Producer: `wait(empty)→wait(mutex)→produce→post(mutex)→post(full)`
5. Consumer: `wait(full)→wait(mutex)→consume→post(mutex)→post(empty)`
6. Create threads, join threads

■ 10. 3 Marks Summary

Producer-Consumer uses 3 semaphores: empty (free slots), full (filled slots), mutex (lock). Producer does: `wait(empty)`, `lock`, `produce`, `unlock`, `post(full)`. Consumer does: `wait(full)`, `lock`, `consume`, `unlock`, `post(empty)`. Compile with `-lpthread`.

Shared Memory Implementation

1. Aim

To implement inter-process communication using POSIX shared memory.

2. Important Concept

Shared Memory = fastest IPC method. Two processes share the SAME memory region.

One process writes to it. Another process reads from it. No copying of data!

Steps: shmget() → shmat() → USE → shmdt() → shmctl(IPC_RMID)

It's like two people writing on the same whiteboard.

3. Program Explanation (Line by Line)

```
// Writer Process
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>

int main() {
    key_t key = 1234;    // unique key for shared memory
    int shmid = shmget(key, 1024, 0666|IPC_CREAT); // CREATE ← KEY
    char *ptr = (char*) shmat(shmid, NULL, 0);      // ATTACH ← KEY
    strcpy(ptr, "Hello from writer!");              // WRITE
    printf("Written: %s\n", ptr);
    shmdt(ptr);    // DETACH
    return 0;
}

// Reader Process
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>

int main() {
    key_t key = 1234;    // SAME key as writer
    int shmid = shmget(key, 1024, 0666); // ATTACH to existing
    char *ptr = (char*) shmat(shmid, NULL, 0);
    printf("Read: %s\n", ptr);            // READ the message
    shmdt(ptr);    // DETACH
    shmctl(shmid, IPC_RMID, NULL); // DESTROY shared memory
    return 0;
}
```

4. How to Read in Viva

4-step process (CREATE → ATTACH → USE → DETACH/DESTROY):

shmget(key, size, flags) — creates or gets shared memory segment

- key: unique ID so both processes find the same memory

- IPC_CREAT flag: creates new segment if it doesn't exist

shmat(shmid, NULL, 0) — attaches to your process's address space, returns pointer

shmdt(ptr) — detaches when done

shmctl(shmid, IPC_RMID, NULL) — removes the segment (cleanup)

■ 5. Expected Output

Writer output:

Written: Hello from writer!

Reader output:

Read: Hello from writer!

■ 6. Frequently Asked Viva Questions

Q1: Why is shared memory the fastest IPC?

A: Data is not copied — both processes directly access the same physical memory.

Q2: What is shmget()?

A: Creates or accesses a shared memory segment. Returns a shared memory ID.

Q3: What is shmat()?

A: Attaches the shared memory segment to the calling process's address space.

Q4: What is IPC_RMID?

A: Flag used with shmctl() to mark the shared memory segment for deletion.

Q5: What is a key in shared memory?

A: A unique integer that identifies the shared memory segment so multiple processes can access the same one.

■ 7. Memory Trick

Shared memory lifecycle = 'CAUD': Create(shmget) → Attach(shmat) → Use(read/write) → Detach(shmdt) → Delete(shmctl IPC_RMID).

■ 8. Common Errors and Fixes

Error: shmget() fails

Fix: Check if key is unique. Use ipcs command to list existing shared memory.

Error: Both processes must use same key

Fix: Key must be identical in writer and reader.

Error: Not calling shmctl IPC_RMID

Fix: Shared memory persists even after program ends. Always clean up.

■ 9. Exam Writing Steps (Algorithm)

1. Include sys/ipc.h and sys/shm.h

2. Define a common key (e.g., 1234)

3. Writer: shmget with IPC_CREAT flag, shmat, write data, shmdt

4. Reader: shmget (no IPC_CREAT), shmat, read data, shmdt

5. Reader: shmctl(shmid, IPC_RMID, NULL) to cleanup

■ 10. 3 Marks Summary

Shared memory is fastest IPC. Flow: shmget() creates segment, shmat() attaches it, use it, shmdt() detaches, shmctl(IPC_RMID) destroys. Both processes use same key. Headers: sys/ipc.h, sys/shm.h.

Banker's Algorithm for Deadlock Avoidance

1. Aim

To implement Banker's Algorithm to check if a system state is safe and avoid deadlock.

2. Important Concept

Banker's Algorithm = check if we can satisfy all processes with available resources.

Key matrices:

Allocation[i][j] = resources of type j currently given to process i

Max[i][j] = maximum resources of type j process i may ever need

Need[i][j] = Max[i][j] - Allocation[i][j] (still needed)

Available[j] = total available resources of type j

SAFE STATE: find an order where all processes can finish (safe sequence).

3. Program Explanation (Line by Line)

```
#include <stdio.h>

#define P 5 // processes
#define R 3 // resource types

int main() {
    int alloc[P][R] = {{0,1,0},{2,0,0},{3,0,2},{2,1,1},{0,0,2}};
    int maxr[P][R] = {{7,5,3},{3,2,2},{9,0,2},{2,2,2},{4,3,3}};
    int avail[R] = {3,3,2};
    int need[P][R], finish[P]={0}, safeSeq[P];

    // Compute Need = Max - Allocation
    for(int i=0;i<P;i++)
        for(int j=0;j<R;j++)
            need[i][j] = maxr[i][j] - alloc[i][j]; // ← KEY FORMULA

    int count=0;
    while(count < P) {
        int found=0;
        for(int i=0;i<P;i++) {
            if(!finish[i]) {
                int ok=1;
                for(int j=0;j<R;j++)
                    if(need[i][j] > avail[j]) { ok=0; break; } // can't satisfy?
                if(ok) {
                    for(int j=0;j<R;j++)
                        avail[j] += alloc[i][j]; // release resources
                    safeSeq[count++] = i;
                    finish[i] = 1;
                }
            }
        }
    }
}
```

```

        found = 1;
    }
}
}
if(!found) { printf("UNSAFE STATE!\n"); return 0; }
}
printf("Safe Sequence: ");
for(int i=0;i<P;i++) printf("P%d ", safeSeq[i]);
}

```

■ 4. How to Read in Viva

Step 1: Compute $Need[i][j] = Max[i][j] - Allocation[i][j]$

Step 2: Safety Algorithm:

- Find unfinished process i where $Need[i] \leq Available$
- If found: add its allocation back to Available, mark finished
- If no such process found: UNSAFE STATE
- Repeat until all processes finish (SAFE) or stuck (UNSAFE)

Key: 'finish' array tracks which processes have completed

Key: 'found' flag detects if we're stuck (no process can proceed)

■ 5. Expected Output

Safe Sequence: P1 P3 P4 P0 P2

(System is in SAFE state)

If no safe sequence exists:

UNSAFE STATE - Deadlock may occur!

■ 6. Frequently Asked Viva Questions

Q1: What is a safe state?

A: A state where there exists at least one safe sequence — an order in which all processes can complete.

Q2: What is Need matrix?

A: $Need[i][j] = Max[i][j] - Allocation[i][j]$. It shows how many more resources each process needs.

Q3: What is the difference between Deadlock Avoidance and Detection?

A: Avoidance prevents deadlock before it happens (Banker's). Detection finds deadlock after it occurs.

Q4: What are the 4 conditions for deadlock?

A: Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait.

Q5: When does Banker's Algorithm say UNSAFE?

A: When no unfinished process can have its Need satisfied by Available resources.

■ 7. Memory Trick

*Banker's = think like a bank. Will you have enough money to give everyone their loans when they ask?
Safe = yes, you can pay everyone. Unsafe = you'll run out of money.*

■ 8. Common Errors and Fixes

Error: Need matrix computed wrong

Fix: $Need = Max - Allocation$. Not the other way around!

Error: Not releasing resources after process finishes

Fix: When process finishes: Available += Allocation of that process.

Error: Infinite loop without found flag

Fix: If no process can be satisfied in a full pass, it's unsafe. Break out.

■ 9. Exam Writing Steps (Algorithm)

1. Initialize Allocation, Max, Available matrices
2. Compute Need = Max - Allocation
3. Safety check: find process where Need <= Available
4. Mark it finished, add its allocation to Available
5. Repeat until all done (SAFE) or stuck (UNSAFE)
6. Print safe sequence

■ 10. 3 Marks Summary

Banker's Algorithm avoids deadlock by checking safety before allocating resources. Need = Max - Allocation. Safety check: find processes whose Need <= Available, simulate completion, repeat. If all can complete = SAFE state. Otherwise = UNSAFE. Does NOT guarantee no deadlock request — it prevents unsafe allocation.

Deadlock Detection Algorithm

1. Aim

To detect whether a deadlock exists in the current system state using the resource allocation graph approach.

2. Important Concept

Deadlock Detection is used AFTER deadlock may have occurred.

Uses Wait-For Graph or resource matrix approach.

Algorithm finds processes that CAN finish with available resources.

If some processes CANNOT finish = DEADLOCK detected among them.

Difference from Banker's: Detection checks CURRENT state. Banker's AVOIDS unsafe states.

3. Program Explanation (Line by Line)

```
#include <stdio.h>

#define P 4
#define R 3

int main() {
    int alloc[P][R] = {{0,1,0},{2,0,0},{3,0,3},{0,1,1}};
    int request[P][R] = {{0,0,0},{2,0,2},{0,0,0},{1,0,0}};
    int avail[R] = {0,0,0};
    int finish[P] = {0,0,0,0};

    // Processes with zero allocation are immediately done
    for(int i=0;i<P;i++) {
        int allzero=1;
        for(int j=0;j<R;j++) if(alloc[i][j]!=0) { allzero=0; break; }
        if(allzero) finish[i]=1;
    }

    int changed=1;
    while(changed) {
        changed=0;
        for(int i=0;i<P;i++) {
            if(!finish[i]) {
                int ok=1;
                for(int j=0;j<R;j++)
                    if(request[i][j] > avail[j]) { ok=0; break; }
                if(ok) {
                    for(int j=0;j<R;j++) avail[j]+=alloc[i][j];
                    finish[i]=1; changed=1;
                }
            }
        }
    }
}
```

```

}

int deadlock=0;
printf("Deadlocked processes: ");
for(int i=0;i<P;i++)
    if(!finish[i]) { printf("P%d ",i); deadlock=1; }
if(!deadlock) printf("None – No Deadlock!");
}

```

■ 4. How to Read in Viva

Key differences from Banker's:

- Uses 'request' (current request) instead of 'need' (max future need)
- Available starts at ACTUAL available, not hypothetical
- Processes with 0 allocation are marked done first
- If finish[i] = 0 at end = process is DEADLOCKED

■ 5. Expected Output

```

Deadlocked processes: P1 P3
(P0 and P2 are not deadlocked – they either finished or had no requests))

```

■ 6. Frequently Asked Viva Questions

Q1: Difference between Deadlock Avoidance and Detection?

A: Avoidance (Banker's) prevents unsafe states before allocation. Detection finds deadlock after it occurs.

Q2: What is the Wait-For Graph?

A: A directed graph where $P1 \rightarrow P2$ means $P1$ is waiting for $P2$. A cycle = deadlock.

Q3: What are 4 conditions for deadlock?

A: Mutual Exclusion, Hold-and-Wait, No Preemption, Circular Wait. ALL four must hold simultaneously.

Q4: How to recover from deadlock?

A: Kill one deadlocked process, preempt resources, or rollback processes.

Q5: What matrix does detection use instead of Need?

A: Request matrix — the current actual request, not the maximum future need.

■ 7. Memory Trick

Detection vs Avoidance: Detection is a DOCTOR (diagnoses after symptoms appear). Avoidance is a VACCINE (prevents the disease before it happens).

■ 8. Common Errors and Fixes

Error: Using Need instead of Request

Fix: Deadlock Detection uses current Request matrix, not Max-Allocation.

Error: Not initializing processes with zero allocation as done

Fix: These processes aren't holding resources, so they're not deadlocked.

Error: Confusing with Banker's Algorithm

Fix: In Detection: Available = actual free resources. In Banker's: Available is simulated.

■ 9. Exam Writing Steps (Algorithm)

1. Initialize Allocation, Request, Available matrices
2. Mark processes with all-zero allocation as finish=1

3. Loop: find unfinished process where Request $\leq$ Available
4. Release its resources: Available $+=$ Allocation
5. Mark it finish=1
6. Repeat until no more found
7. Any process with finish=0 is DEADLOCKED

■ 10. 3 Marks Summary

Deadlock Detection runs on current state to find deadlocked processes. Uses Request (not Need). Processes with zero allocation are immediately done. Loop: if Request[i] $\leq$ Available, simulate completion, release resources. Remaining unfinished processes = deadlocked. Recovery: kill process or preempt resources.

FIFO Page Replacement Algorithm

1. Aim

To implement FIFO page replacement and calculate the number of page faults.

2. Important Concept

FIFO = First In, First Out. The page that has been in memory LONGEST is replaced first.

Like a queue: first to enter is first to leave.

Page Fault = required page is NOT in memory frames → load it (increase fault count).

If page already in memory = Page HIT (no fault).

3. Program Explanation (Line by Line)

```
#include <stdio.h>

int main() {
    int pages[] = {7,0,1,2,0,3,0,4,2,3};
    int n = 10; // number of page references
    int frames = 3; // number of memory frames
    int frame[3] = {-1,-1,-1}; // -1 = empty frame
    int faults = 0, ptr = 0; // ptr = next frame to replace (FIFO)

    for(int i=0; i<n; i++) {
        int found = 0;
        // Check if page already in frames (page hit)
        for(int j=0; j<frames; j++)
            if(frame[j] == pages[i]) { found=1; break; }

        if(!found) { // PAGE FAULT! ← KEY
            frame[ptr] = pages[i]; // replace oldest frame
            ptr = (ptr+1) % frames; // move FIFO pointer
            faults++;
            printf("Page fault for page %d\n", pages[i]);
        }
    }
    printf("Total page faults: %d\n", faults);
    return 0;
}
```

4. How to Read in Viva

frame[] = current pages in memory (-1 = empty)

ptr = FIFO pointer, points to the OLDEST frame (next to replace)

For each page reference:

1. Check if page is already in any frame (page hit)
2. If NOT found: page fault → replace frame[ptr], advance ptr in circular fashion

3. If found: page hit, do nothing, no fault

ptr moves circularly: $(ptr+1) \% frames$ — always replaces oldest page

■ 5. Expected Output

Reference: 7 0 1 2 0 3 0 4 2 3

Frames=3

7: [7, -, -] FAULT

0: [7, 0, -] FAULT

1: [7, 0, 1] FAULT

2: [2, 0, 1] FAULT (replace 7)

0: [2, 0, 1] HIT

3: [2, 3, 1] FAULT (replace 0)

Total page faults: 6

■ 6. Frequently Asked Viva Questions

Q1: What is a page fault?

A: When the required page is not in any memory frame, OS must load it from disk. Each load = 1 page fault.

Q2: What does FIFO replace?

A: The page that has been in memory the longest (first to arrive = first to leave).

Q3: What is Belady's Anomaly?

A: In FIFO, increasing the number of frames can sometimes INCREASE page faults (unusual behavior).

Q4: Which algorithm does NOT suffer from Belady's Anomaly?

A: LRU (Least Recently Used) and OPT (Optimal) do not suffer from Belady's Anomaly.

Q5: Difference between FIFO, LRU, LFU?

A: FIFO: replace oldest. LRU: replace least recently used. LFU: replace least frequently used.

■ 7. Memory Trick

FIFO = bus queue. First passenger to board is first to exit. The ptr pointer is like the conductor marking who arrived first. When bus is full, first passenger must get off!

■ 8. Common Errors and Fixes

Error: ptr not moving circularly

Fix: Use $ptr = (ptr+1) \% frames$ to wrap around.

Error: Counting hit as fault

Fix: Only increment faults when page is NOT in any frame.

Error: Frames initialized to 0 not -1

Fix: Use -1 to mark empty frames; page 0 would be confused with empty frame.

■ 9. Exam Writing Steps (Algorithm)

1. Initialize frames[] = -1 (empty), ptr=0, faults=0
2. For each page in reference string:
3. Check if page is in any frame (page hit?)
4. If NO: fault! Replace frame[ptr], $ptr=(ptr+1)\%frames$, faults++
5. Print frame state after each step
6. Print total faults

■ 10. 3 Marks Summary

FIFO replaces the page that entered memory first. Uses a circular pointer. Page fault occurs when page not in frames. Susceptible to Belady's Anomaly. Simple to implement. Formula: page faults count how many times a page had to be loaded from disk.

LRU Page Replacement Algorithm

1. Aim

To implement Least Recently Used (LRU) page replacement algorithm.

2. Important Concept

LRU = Least Recently Used. Replace the page that was used LEAST RECENTLY (longest ago).

Based on the idea: if a page hasn't been used recently, it probably won't be used soon.

Better than FIFO. Does NOT suffer from Belady's Anomaly.

Uses 'last_used[]' array to track when each frame was last accessed.

3. Program Explanation (Line by Line)

```
#include <stdio.h>

int main() {
    int pages[] = {7,0,1,2,0,3,0,4,2,3};
    int n=10, frames=3;
    int frame[3]={-1, -1, -1};
    int last_used[3]={0,0,0}; // time of last use for each frame
    int faults=0, time=0;

    for(int i=0; i<n; i++) {
        time++;
        int found=-1;
        for(int j=0; j<frames; j++)
            if(frame[j]==pages[i]) { found=j; break; }

        if(found != -1) { // PAGE HIT
            last_used[found] = time; // update last used time
        } else { // PAGE FAULT
            // Find LRU frame: frame with smallest last_used
            int lru=0;
            for(int j=1; j<frames; j++)
                if(last_used[j] < last_used[lru]) lru=j; // ← KEY
            frame[lru] = pages[i]; // replace LRU frame
            last_used[lru] = time;
            faults++;
        }
    }

    printf("Page faults: %d\n", faults);
    return 0;
}
```

4. How to Read in Viva

Key concept: last_used[] tracks WHEN each frame was last accessed
On PAGE HIT: update last_used[found] = current time (refresh its 'freshness')
On PAGE FAULT: find frame with MINIMUM last_used value (= used longest ago)
Replace that frame with new page, set its last_used = current time
This is the LRU frame = least recently used = replace it first

■ 5. Expected Output

```
Reference: 7 0 1 2 0 3 0 4 2 3
7: [7, -, -] FAULT    (time=1)
0: [7, 0, -] FAULT    (time=2)
1: [7, 0, 1] FAULT    (time=3)
2: [2, 0, 1] FAULT    (replace 7, LRU at time=1)
0: [2, 0, 1] HIT      (update last_used[1]=5)
Total page faults: 4
```

■ 6. Frequently Asked Viva Questions

Q1: Why is LRU better than FIFO?

A: LRU is based on actual usage history (recency). FIFO just uses age in memory, ignoring usage.

Q2: Does LRU suffer from Belady's Anomaly?

A: No, LRU is a stack algorithm and does not suffer from Belady's Anomaly.

Q3: What does last_used array store?

A: The timestamp of when each frame was last accessed. Lower value = used longer ago = LRU candidate.

Q4: LRU vs LFU?

A: LRU: replaces least RECENTLY used. LFU: replaces least FREQUENTLY used (lowest access count).

Q5: Is OPT better than LRU?

A: Yes, OPT (Optimal) gives minimum page faults but requires future knowledge, so it's theoretical only.

■ 7. Memory Trick

LRU = 'Least Recently Used' = find the page with the OLDEST last-use timestamp. Think of it as: among all your friends in memory, kick out the one you haven't talked to the longest!

■ 8. Common Errors and Fixes

Error: Updating last_used only on fault, not on hit

Fix: On HIT too, update last_used! That's the whole point of LRU.

Error: Finding minimum vs maximum

Fix: LRU = minimum last_used value = used longest ago = replace this one.

Error: Not incrementing time for each reference

Fix: Time must tick for every page reference, hit or fault.

■ 9. Exam Writing Steps (Algorithm)

1. Initialize frames=-1, last_used=0, time=0, faults=0
2. For each page: increment time
3. Check if page in frames (page hit?)
4. If HIT: update last_used[found] = time
5. If FAULT: find frame with min last_used, replace it, set last_used=time, faults++
6. Print total faults

■ 10. 3 Marks Summary

LRU replaces the page used least recently. Tracks access time with last_used[]. On hit: refresh last_used. On fault: find min last_used frame and replace it. Better than FIFO. No Belady's Anomaly. More overhead than FIFO but better performance.

LFU Page Replacement Algorithm

1. Aim

To implement Least Frequently Used (LFU) page replacement algorithm.

2. Important Concept

LFU = Least Frequently Used. Replace the page with the LOWEST usage COUNT.

The idea: if a page was rarely accessed in the past, it's probably not important.

Tracks COUNT (how many times accessed), not TIME (when last accessed).

Tie-break: if two pages have the same count, replace the one that's been there longer.

3. Program Explanation (Line by Line)

```
#include <stdio.h>

int main() {
    int pages[] = {1,2,3,4,1,2,5,1,2,3,4,5};
    int n=12, frames=3;
    int frame[3]={-1, -1, -1};
    int freq[3]={0,0,0};    // frequency/count for each frame
    int age[3]={0,0,0};    // for tie-breaking: higher age = older
    int faults=0, timer=0;

    for(int i=0; i<n; i++) {
        timer++;
        int found=-1;
        for(int j=0; j<frames; j++)
            if(frame[j]==pages[i]) { found=j; break; }

        if(found != -1) {          // HIT: increment frequency
            freq[found]++;
        } else {                  // FAULT
            // Find empty frame first
            int lfu=-1;
            for(int j=0; j<frames; j++)
                if(frame[j]==-1) { lfu=j; break; }

            if(lfu==-1) { // no empty frame: find LFU frame
                lfu=0;
                for(int j=1; j<frames; j++)
                    if(freq[j]<freq[lfu] ||
                       (freq[j]==freq[lfu] && age[j]<age[lfu]))
                        lfu=j;    // ← KEY: lowest freq, older on tie
            }

            frame[lfu]=pages[i]; freq[lfu]=1; age[lfu]=timer;
            faults++;
        }
    }
}
```

```

    }
}
printf("Page faults: %d\n", faults);
return 0;
}

```

■ 4. How to Read in Viva

freq[] = how many times each frame has been accessed

age[] = when each frame was loaded (for tie-breaking)

On PAGE HIT: increment freq[found]++ only (access count goes up)

On PAGE FAULT:

1. Use empty frame if available
2. Else: find frame with minimum freq (tie: use older one)
3. Replace that frame, reset freq=1, update age

■ 5. Expected Output

```

Reference: 1 2 3 4 1 2 5 1 2 3 4 5
1: [1, -, -] FAULT  freq=[1,0,0]
2: [1,2, -] FAULT  freq=[1,1,0]
3: [1,2,3] FAULT  freq=[1,1,1]
4: [1,2,4] FAULT  (replace 3, LFU tie→oldest)
1: [1,2,4] HIT    freq=[2,1,1]
Total page faults: 8

```

■ 6. Frequently Asked Viva Questions

Q1: LFU vs LRU difference?

A: LFU tracks frequency (count of accesses). LRU tracks recency (time of last access).

Q2: What is the disadvantage of LFU?

A: A page used many times early but no longer needed stays in memory (frequency doesn't decrease).

Q3: How is tie-breaking done in LFU?

A: Among pages with same frequency, replace the one loaded earliest (oldest page).

Q4: LFU vs FIFO?

A: LFU considers how often a page is used. FIFO just considers arrival order.

Q5: Which algorithm is theoretically optimal?

A: OPT (Optimal) — replace page not used for longest time in future. Impractical as it needs future knowledge.

■ 7. Memory Trick

LFU = 'Least Fan Club'. The page with the fewest 'fans' (least accesses) gets kicked out. Count the fans — kick out the least popular page!

■ 8. Common Errors and Fixes

Error: Resetting frequency to 0 on replacement

Fix: Set freq=1 when loading new page (it's been accessed once, right now).

Error: Not handling empty frames

Fix: Check for empty frames (-1) before looking for LFU page.

Error: No tie-breaking logic

Fix: When frequencies are equal, use age or LRU to decide which to replace.

■ 9. Exam Writing Steps (Algorithm)

1. Initialize frames=-1, freq=0, faults=0
2. For each page: check if in frames (HIT or FAULT)
3. HIT: increment freq[found]
4. FAULT: find empty frame or min-freq frame
5. Load page, set freq=1, record age/time
6. Print total faults

■ 10. 3 Marks Summary

LFU replaces page with lowest access frequency. Tracks freq[] count. On hit: freq++. On fault: replace min freq (tie: oldest). Disadvantage: old frequently-used pages stay even if no longer needed. Compare: FIFO=oldest in, LRU=least recently used, LFU=least frequently used.

Bell-LaPadula Security Model

1. Aim

To implement and demonstrate the Bell-LaPadula confidentiality security model.

2. Important Concept

Bell-LaPadula Model = CONFIDENTIALITY model. Protects secret information.

Security levels: Top Secret > Secret > Confidential > Unclassified

Two key rules:

1. No Read Up (Simple Security Property): You can't read HIGHER classification than yours
2. No Write Down (*-Property): You can't write to LOWER classification than yours

Think: A Private soldier cannot read Top Secret files. A General cannot leak secrets to public files.

3. Program Explanation (Line by Line)

```
#include <stdio.h>
#include <string.h>

// Security levels: 0=Unclassified, 1=Confidential, 2=Secret, 3=Top Secret
char* level_name[] = {"Unclassified","Confidential","Secret","Top Secret"};

void check_read(char* user, int user_level, char* object, int obj_level) {
    printf("\nRead: %s (level %d) reads %s (level %d): ",
           user, user_level, object, obj_level);
    if(user_level >= obj_level) // No Read Up rule ← KEY
        printf("ALLOWED ✓ (Simple Security Property)");
    else
        printf("DENIED ✗ (No Read Up violated)");
}

void check_write(char* user, int user_level, char* object, int obj_level) {
    printf("\nWrite: %s (level %d) writes to %s (level %d): ",
           user, user_level, object, obj_level);
    if(user_level <= obj_level) // No Write Down rule ← KEY
        printf("ALLOWED ✓ (*-Property)");
    else
        printf("DENIED ✗ (No Write Down violated)");
}

int main() {
    check_read("General",3,"Top Secret File",3);    // allowed
    check_read("Private",0,"Secret File",2);         // denied
    check_write("General",3,"Top Secret File",3);    // allowed
    check_write("General",3,"Public File",0);        // denied
    return 0;
}
```

■ 4. How to Read in Viva

Rule 1 - Simple Security Property (No Read Up):

User can read object only if $\text{user_level} \geq \text{object_level}$

Cannot read ABOVE your clearance level

Rule 2 - *-Property (No Write Down / Star Property):

User can write to object only if $\text{user_level} \leq \text{object_level}$

Cannot write to LOWER security level (to prevent leaking secrets)

This model focuses on CONFIDENTIALITY — keeping secrets secret.

■ 5. Expected Output

Read: General (level 3) reads Top Secret File (level 3): ALLOWED

Read: Private (level 0) reads Secret File (level 2): DENIED

Write: General (level 3) writes to Top Secret File (level 3): ALLOWED

Write: General (level 3) writes to Public File (level 0): DENIED

■ 6. Frequently Asked Viva Questions

Q1: What does Bell-LaPadula protect?

A: CONFIDENTIALITY — it prevents unauthorized reading of sensitive information.

Q2: What is the No Read Up rule?

A: A subject cannot read an object with a higher security classification than their own.

Q3: What is the No Write Down rule (*-property)?

A: A subject cannot write to an object with a lower security classification (prevents leaking secrets downward).

Q4: What is Biba model?

A: Biba model is the opposite — it protects INTEGRITY (No Write Up, No Read Down).

Q5: Give a real-life example of Bell-LaPadula.

A: Military: a soldier with Secret clearance cannot read Top Secret documents and cannot write to Unclassified files.

■ 7. Memory Trick

BLP = 'Bag Locked Properly'. Read UP = locked (denied). Write DOWN = locked (denied). Think: secrets flow UP, never DOWN. Generals can't contaminate public files!

■ 8. Common Errors and Fixes

Error: Confusing read and write rules

Fix: READ: $\text{user_level} \geq \text{object_level}$ to allow. WRITE: $\text{user_level} \leq \text{object_level}$ to allow.

Error: Confusing BLP with Biba

Fix: BLP = Confidentiality (no read up, no write down). Biba = Integrity (no write up, no read down).

Error: Security levels not ordered

Fix: Always: Unclassified(0) < Confidential(1) < Secret(2) < Top Secret(3).

■ 9. Exam Writing Steps (Algorithm)

1. Define security levels as integers (0=lowest, 3=highest)
2. For READ: allow only if $\text{user_level} \geq \text{object_level}$
3. For WRITE: allow only if $\text{user_level} \leq \text{object_level}$
4. Test with different user and object level combinations
5. Print ALLOWED or DENIED with reason

■ 10. 3 Marks Summary

Bell-LaPadula Model ensures CONFIDENTIALITY. Two rules: No Read Up (user_level $\geq$ object_level to read) and No Write Down (user_level $\leq$ object_level to write). Levels: Unclassified < Confidential < Secret < Top Secret. Military-grade secrecy model.

Biba Integrity Model

1. Aim

To implement and demonstrate the Biba integrity security model.

2. Important Concept

Biba Model = INTEGRITY model. Protects data from being corrupted.

It is the OPPOSITE of Bell-LaPadula.

Two key rules:

1. No Write Up (Simple Integrity Property): Low integrity user cannot write to HIGH integrity object
2. No Read Down (*-Integrity Property): High integrity user cannot read from LOW integrity object

Think: A junior employee cannot modify important company policy. A CEO shouldn't read untrusted rumors.

3. Program Explanation (Line by Line)

```
#include <stdio.h>

// Integrity levels: 0=Low, 1=Medium, 2=High
char* level_name[] = {"Low", "Medium", "High"};

void check_write(char* user, int user_level, char* object, int obj_level) {
    printf("\nWrite: %s (%s) writes to %s (%s): ",
           user, level_name[user_level], object, level_name[obj_level]);
    if(user_level >= obj_level) // No Write Up: can write to SAME or LOWER ← KEY
        printf("ALLOWED ✓");
    else
        printf("DENIED ✗ (Cannot write up in Biba)");
}

void check_read(char* user, int user_level, char* object, int obj_level) {
    printf("\nRead: %s (%s) reads %s (%s): ",
           user, level_name[user_level], object, level_name[obj_level]);
    if(user_level <= obj_level) // No Read Down: can read SAME or HIGHER ← KEY
        printf("ALLOWED ✓");
    else
        printf("DENIED ✗ (Cannot read down in Biba)");
}

int main() {
    check_write("Manager", 2, "Policy Doc", 2); // allowed: same level
    check_write("Intern", 0, "Policy Doc", 2); // denied: write up
    check_read("Manager", 2, "Rumor File", 0); // denied: read down
    check_read("Intern", 0, "Policy Doc", 2); // allowed: read up ok
    return 0;
}
```

■ 4. How to Read in Viva

WRITE rule (No Write Up):

$\text{user_level} \geq \text{object_level} \rightarrow \text{ALLOWED}$

Low integrity user CANNOT write to High integrity object

Prevents contamination of high-integrity data

READ rule (No Read Down):

$\text{user_level} \leq \text{object_level} \rightarrow \text{ALLOWED}$

High integrity user CANNOT read from Low integrity source

Prevents corruption from low-quality/untrusted data

■ 5. Expected Output

Write: Manager (High) writes to Policy Doc (High): ALLOWED

Write: Intern (Low) writes to Policy Doc (High): DENIED

Read: Manager (High) reads Rumor File (Low): DENIED

Read: Intern (Low) reads Policy Doc (High): ALLOWED

■ 6. Frequently Asked Viva Questions

Q1: What does Biba protect?

A: INTEGRITY — it prevents unauthorized modification of trusted data.

Q2: How does Biba differ from Bell-LaPadula?

A: BLP: Confidentiality (no read up, no write down). Biba: Integrity (no write up, no read down).

Q3: What is the No Write Up rule in Biba?

A: A low-integrity subject cannot write to a higher-integrity object (prevents data corruption).

Q4: Real-life example of Biba?

A: A junior employee cannot modify the company's master database. A CEO won't read unverified social media rumors.

Q5: Can Biba and BLP be combined?

A: Yes, some systems use both to protect both confidentiality AND integrity simultaneously.

■ 7. Memory Trick

BIBA = 'Big Integrity, Block Attacks'. No Write UP (juniors can't corrupt important data), No Read DOWN (experts don't read garbage). BIBA is BLP flipped upside down!

■ 8. Common Errors and Fixes

Error: Confusing read/write rules with BLP

Fix: Biba: write requires $\text{user} \geq \text{object}$. Read requires $\text{user} \leq \text{object}$. BLP is the opposite for confidentiality.

Error: Mixing up Biba and BLP

Fix: BLP: NO Read Up, NO Write Down. Biba: NO Write Up, NO Read Down.

Error: Wrong integrity level direction

Fix: Higher level = MORE trusted/important. Low level = less trusted.

■ 9. Exam Writing Steps (Algorithm)

1. Define integrity levels (0=Low, 1=Medium, 2=High)

2. WRITE check: allow if $\text{user_level} \geq \text{object_level}$

3. READ check: allow if $\text{user_level} \leq \text{object_level}$

4. Test with various level combinations

5. Print ALLOWED or DENIED with reason

■ 10. 3 Marks Summary

Biba Model ensures INTEGRITY. Two rules: No Write Up ($\text{user_level} \geq \text{object_level}$ to write) and No Read Down ($\text{user_level} \leq \text{object_level}$ to read). Opposite of Bell-LaPadula. BLP=Confidentiality (secrets safe), Biba=Integrity (data correct). Both may be used together in real systems.

Quick Revision Sheet

ALL EXPERIMENT AIMS AT A GLANCE

#	Experiment	Aim (One Line)
1	UNIX Commands	Learn basic shell commands for file/process management
2	Fork()	Create child process; fork() returns 0 to child, PID to parent
3	Wait()	Parent waits for child to finish; prevents zombie processes
4	Exec()	Replace current process with new program; code after exec() never runs
5	Stat()	Read file metadata (size, inode, permissions) using struct stat
6	Readdir()	List directory entries using opendir/readdir/closedir
7	Round Robin	Equal time quantum for each process; circular execution; no starvation
8	SJF	Shortest burst time runs first; optimal average WT; may starve long jobs
9	FCFS	First arrival runs first; simple but convoy effect possible
10	Priority	Highest priority (lowest number) runs first; starvation fixed by aging
11	Producer-Consumer	Semaphores synchronize shared buffer: empty, full, mutex
12	Shared Memory	Fastest IPC: shmget→shmat→use→shmdt→shmctl(IPC_RMID)
13	Banker's Algorithm	Deadlock avoidance: check safety before allocation using Need matrix
14	Deadlock Detection	Find deadlocked processes using Request matrix; finish[] tracking
15	FIFO Page Replace	Replace oldest page (circular pointer); suffers Belady's anomaly
16	LRU Page Replace	Replace least recently used page (min last_used); no Belady's
17	LFU Page Replace	Replace least frequently accessed page (min freq count)
18	Bell-LaPadula	Confidentiality: No Read Up, No Write Down
19	Biba Model	Integrity: No Write Up, No Read Down (opposite of BLP)

IMPORTANT FORMULAS

Waiting Time (WT)	$WT = \text{Turnaround Time} - \text{Burst Time}$
Turnaround Time (TAT)	$TAT = \text{Completion Time} - \text{Arrival Time} (= WT + BT \text{ if } AT=0)$
Average WT / TAT	Sum of all WT (or TAT) divided by number of processes
Need (Banker's)	$\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$
Page Fault Rate	$\text{Faults} / \text{Total Page References} * 100\%$
File Permissions	chmod: r=4, w=2, x=1. Example: 755 = rwxr-xr-x
RR Remaining Time	$\text{remaining}[i] = \text{remaining}[i] - \text{quantum} \text{ (if remaining} > \text{quantum)}$

IMPORTANT UNIX COMMANDS

ls -l	Long list with permissions
pwd	Print working directory
cd /path	Change directory
mkdir dir	Create directory
rm file	Remove file
cp src dst	Copy file
mv src dst	Move/rename file
cat file	Show file content
chmod 755 f	Set permissions

<code>grep pat file</code>	Search in file
<code>ps aux</code>	Show all processes
<code>kill PID</code>	Kill a process
<code>man cmd</code>	Show manual for command
<code>who</code>	Show logged users
<code>wc -l file</code>	Count lines in file
<code>find . -name f</code>	Find file by name

MOST ASKED VIVA QUESTIONS

Q: What does fork() return?	A: 0 to child, child's PID to parent, -1 on error
Q: What is a zombie process?	A: A finished process whose parent hasn't called wait() yet
Q: What is a page fault?	A: Required page not in memory — must load from disk
Q: What is Belady's Anomaly?	A: FIFO: more frames can cause MORE page faults
Q: 4 conditions for deadlock?	A: Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait
Q: Banker's Need formula?	A: $\text{Need} = \text{Max} - \text{Allocation}$
Q: Bell-LaPadula rules?	A: No Read Up, No Write Down (confidentiality)
Q: Biba rules?	A: No Write Up, No Read Down (integrity)
Q: What is Aging in scheduling?	A: Gradually increase priority of waiting processes to prevent starvation
Q: What does exec() do?	A: Replaces current process with new program. Code after exec() won't run.
Q: Which scheduling has min avg WT?	A: SJF (Shortest Job First) is optimal for non-preemptive
Q: What is shmget()?	A: Creates or accesses a shared memory segment
Q: Why 3 semaphores in Prod-Cons?	A: empty=free slots, full=filled slots, mutex=prevents race condition
Q: What does stat() return?	A: 0 on success, -1 on error. Fills struct stat with file metadata.

QUICK DIFFERENCES

Scheduling Algorithms Comparison:

Feature	FCFS	SJF	Round Robin	Priority
Order	Arrival time	Burst time (min first)	Time quantum	Priority number
Type	Non-preemptive	Non-preemptive	Preemptive (usually)	Both types
Starvation	No	Yes (long jobs)	No	Yes (low priority)
Avg WT	Worst	Best (optimal)	Depends on quantum	Depends
Problem	Convoy effect	Starvation	Context switch overhead	Starvation
Best for	Batch systems	Short jobs	Time-sharing OS	Real-time systems

Page Replacement Algorithms Comparison:

Feature	FIFO	LRU	LFU
Replaces	Oldest page (longest in memory)	Least recently used page	Least frequently used page
Tracks	Arrival order (circular ptr)	Time of last access	Access count (frequency)
Belady's Anomaly	YES — suffers from it	NO — stack algorithm	NO
Overhead	Low (simple pointer)	Medium (timestamps)	Higher (counters + tie-break)
Performance	Moderate	Good	Good (but old hot pages stuck)

Deadlock Avoidance vs Detection:

Feature	Deadlock Avoidance (Banker's)	Deadlock Detection
When it runs	Before allocation (proactive)	After deadlock may occur (reactive)
Algorithm	Banker's Algorithm	Resource Allocation Graph / Matrix
Key matrix	Need = Max - Allocation	Request (current actual requests)
Goal	Prevent unsafe state	Find which processes are deadlocked
Action on problem	Deny the resource request	Kill process / preempt resources
Overhead	High (checks every request)	Lower (runs periodically)

Bell-LaPadula vs Biba Model:

Feature	Bell-LaPadula (BLP)	Biba Model
Protects	Confidentiality (secrecy)	Integrity (data correctness)
Read rule	No Read Up: user_level $\geq$ object_level to read	No Read Down: user_level $\leq$ object_level to read
Write rule	No Write Down: user_level $\leq$ object_level to write	No Write Up: user_level $\geq$ object_level to write
Direction	Secrets flow UP, never down	Trusted data: no low-level writes go UP
Example	Military: Pvt can't read Top Secret	Junior can't modify company policy DB
Relationship	Opposite rules to Biba	Opposite rules to BLP

EXAM TIP: For scheduling — always show the Gantt chart if asked. For page replacement — show the frame table step by step. For security models — draw the level diagram. Good luck! ■